RTD_SIM Project Structure Analysis & Phase 6.2 Integration Plan

📊 Current Project Structure

Core Components (32,000+ lines total)

```
RTD_SIM/
├── agent/              (4,558 lines) - BDI agents, stories, social networks
├── analytics/          (4,043 lines) - SHAP, policy impact, validation
├── events/             (1,435 lines) - NEW: Redis pub/sub, spatial index
├── simulation/         (9,500+ lines) - Main simulation engine
│   ├── execution/        - Simulation loop, policies, SD integration
│   ├── infrastructure/   - Charging, grid, depots, pricing
│   ├── routing/          - Route planning, diversity
│   ├── spatial/          - Congestion, graph, metrics, router
│   └── config/           - All configuration files
├── scenarios/          (1,376 lines) - Dynamic policies, scenario manager
├── ui/                 (5,500+ lines) - Streamlit interface, tabs, widgets
├── visualiser/         (1,450 lines) - Animation, visualization
└── environmental/      (1,416 lines) - Weather, emissions, air quality
```

Entry Points

- ✅ **`ui/streamlit_app.py`** (292 lines) - Main entry (ACTIVE)
- ⚠️ **`main.py`** (206 lines) - Old CLI entry (DEPRECATED)
- ✅ **`headless_simulation_runner.py`** (666 lines) - Batch runs (ACTIVE)

Test Files (Currently in root - should move to debug/)

- `test_event_system.py` (234 lines) - Phase 6.1 tests

- `terminal_1_subscriber.py` (139 lines) - Multi-terminal test
- `terminal_2_publisher.py` (243 lines) - Multi-terminal test
- `debug_spatial_test.py` (98 lines) - Spatial filtering debug

🎯 Phase 6.2 Integration Plan

Goal: Integrate R-tree spatial index into SpatialEventBus for O(log N) queries

Files to Modify:

1. events/event_bus.py (528 lines) ⭐ PRIMARY

Current:

```python
class SpatialEventBus(EventBus):
    def __init__(self):
        self.agent_locations = {}  # Dict-based (O(N) queries)

    def _is_event_perceivable(self, agent_id, event):
        # Brute force: check distance to every agent
        for agent_id, location in self.agent_locations.items():
            distance = haversine(...)
```

After Phase 6.2:

```python
```

```python
from .spatial_index import SpatialIndex

class SpatialEventBus(EventBus):
    def __init__(self):
        self.spatial_index = SpatialIndex()  # R-tree (O(log N) queries)

    def _find_perceiving_agents(self, event):
        # Fast: query only nearby agents
        nearby = self.spatial_index.query_radius(
            event.spatial.latitude,
            event.spatial.longitude,
            event.spatial.radius_km
        )
        return [obj.object_id for obj in nearby]
```

Changes needed:

- Replace `agent_locations` dict with `spatial_index`
- Update `register_agent()` to insert into index
- Update `update_agent_location()` to update index
- Update `_is_event_perceivable()` to use radius query

---

2. simulation/execution/simulation_loop.py (837 lines)

Purpose: Initialize event bus in main simulation loop

Current (approximate):

```python
```

```
def run_simulation(...):
    # No event bus initialization
    ...
```

After Phase 6.2:

```python
from events.event_bus import SpatialEventBus

def run_simulation(...):
    # Initialize event bus
    event_bus = SpatialEventBus()
    event_bus.start_listening()

    # Register all agents
    for agent in agents:
        event_bus.register_agent(
            agent_id=agent.agent_id,
            lat=agent.latitude,
            lon=agent.longitude,
            perception_radius_km=5.0
        )

    # Pass to components that need it
    ...

    # Cleanup
    event_bus.close()
```

---

3. agent/story_driven_agent.py (369 lines)

Purpose: Subscribe agents to relevant events

Add method:

```python
def subscribe_to_events(self, event_bus):
    """Subscribe agent to perceivable events."""
    from events.event_types import EventType

    def handle_infrastructure_failure(event):
        # Update beliefs
        self.beliefs['infrastructure_failures'].append(event)
        # Trigger replanning (Phase 7)
        self.replan_needed = True

    def handle_policy_change(event):
        # Update beliefs
        self.beliefs['policies'][event.payload['parameter']] = event.payload['new_value']
        # Recalculate costs
        self.recalculate_mode_costs()

    # Subscribe
    event_bus.subscribe_spatial(self.agent_id, EventType.INFRASTRUCTURE_FAILURE,
                                handle_infrastructure_failure)
    event_bus.subscribe_spatial(self.agent_id, EventType.POLICY_CHANGE,
                                handle_policy_change)
```

---

4. scenarios/dynamic_policy_engine.py (732 lines)

Purpose: Publish policy change events

Add to policy trigger methods:

```python
def _trigger_ev_subsidy(self, current_adoption):
    """Trigger EV subsidy policy."""
    from events.event_bus import EventBus
    from events.event_types import PolicyChangeEvent

    old_value = self.current_policies.get('ev_subsidy', 0)
    new_value = 7000  # New subsidy

    # Update policy
    self.current_policies['ev_subsidy'] = new_value

    # Publish event (if bus available)
    if hasattr(self, 'event_bus') and self.event_bus:
        event = PolicyChangeEvent(
            parameter='ev_subsidy',
            old_value=old_value,
            new_value=new_value,
            lat=56.0,  # Scotland center
            lon=-3.5,
            radius_km=200.0  # Nationwide
        )
        self.event_bus.publish(event)
```

---

5. simulation/infrastructure/infrastructure_manager.py (459 lines)

Purpose: Publish infrastructure failure events

Add to failure handling:

```python
def handle_charging_station_failure(self, station_id, reason):
    """Handle charging station failure."""
    from events.event_types import InfrastructureFailureEvent

    station = self.charging_stations[station_id]

    # Mark as unavailable
    station.available = False

    # Publish event (if bus available)
    if hasattr(self, 'event_bus') and self.event_bus:
        event = InfrastructureFailureEvent(
            infrastructure_type='charging_station',
            infrastructure_id=station_id,
            failure_reason=reason,
            lat=station.latitude,
            lon=station.longitude,
            radius_km=5.0,
            estimated_duration_min=60
        )
        self.event_bus.publish(event)
```

📋 Phase 6.2 Checklist

Step 1: Integrate Spatial Index into Event Bus

- [ ] Update `events/event_bus.py` to use `SpatialIndex`
- [ ] Replace dict-based agent tracking with R-tree
- [ ] Update `register_agent()`, `update_agent_location()`, `remove_agent()`
- [ ] Add `_find_perceiving_agents()` method using radius query
- [ ] Test with existing multi-terminal tests

Step 2: Wire Up Simulation Loop

- [ ] Initialize `SpatialEventBus` in `simulation_loop.py`
- [ ] Register all agents after creation
- [ ] Update agent positions on movement
- [ ] Pass event bus to policy engine and infrastructure manager
- [ ] Ensure proper cleanup on shutdown

Step 3: Connect Policy Engine

- [ ] Add event bus reference to `DynamicPolicyEngine`
- [ ] Publish `PolicyChangeEvent` on each policy trigger
- [ ] Add spatial metadata (lat/lon/radius) for each policy

Step 4: Connect Infrastructure Manager

- [ ] Add event bus reference to `InfrastructureManager`
- [ ] Publish `InfrastructureFailureEvent` on failures
- [ ] Include station location and estimated recovery time

Step 5: Enable Agent Perception

- [ ] Add `subscribe_to_events()` method to agents
- [ ] Subscribe agents in simulation initialization
- [ ] Store perceived events in agent beliefs

- [ ] Log perception events for debugging

Step 6: Test End-to-End

- [ ] Policy change → agents perceive → beliefs updated
- [ ] Infrastructure failure → nearby agents perceive
- [ ] Distant agents don't perceive local events
- [ ] Performance: 1000 agents, 100 events/min

🧹 Cleanup Tasks

Move Test Files to debug/

```bash
mv test_event_system.py debug/
mv terminal_1_subscriber.py debug/
mv terminal_2_publisher.py debug/
mv debug_spatial_test.py debug/
```

Remove/Deprecate Old Files

```bash
# Deprecate old CLI
echo "# DEPRECATED - Use: streamlit run ui/streamlit_app.py" | cat - main.py > temp && mv temp main.py

# Remove old event bus backup
rm simulation/event_bus_OLD_BACKUP.py  # After confirming nothing uses it
```

📊 Performance Targets

Metric	Current	Phase 6.2 Target
Agent registration	N/A	<1ms per agent
Event delivery (10 agents)	0.5ms	0.5ms (no change)
Event delivery (1000 agents)	50ms	3ms (17× faster)
Position updates	N/A	<0.5ms per agent
Memory overhead	Baseline	+5MB for index

🎯 Integration Priority

High Priority (Must Have):

1. ✅ Spatial index in event bus (performance)
2. ✅ Simulation loop initialization (required)
3. ✅ Agent subscription (perception)

Medium Priority (Should Have): 4. 🟡 Policy engine events (valuable) 5. 🟡 Infrastructure failure events (realistic)

Low Priority (Nice to Have): 6. 🟢 Weather events (environmental/weather_api.py) 7. 🟢 Grid stress events (infrastructure/grid/grid_capacity.py)

🚀 Recommended Sequence

Day 1: Core Integration

1. Update `events/event_bus.py` with spatial index
2. Test with existing terminal tests
3. Verify performance improvement

Day 2: Simulation Wiring

1. Update `simulation_loop.py` initialization
2. Register agents after creation
3. Test with small simulation (10 agents, 10 steps)

Day 3: Agent Perception

1. Add agent subscription methods
2. Test event perception in simulation
3. Verify beliefs updated correctly

Day 4: Policy & Infrastructure

1. Add event publishing to policy engine
2. Add event publishing to infrastructure manager
3. End-to-end test: policy change → agents respond

Day 5: Testing & Cleanup

1. Performance testing (1000 agents)

2. Move test files to debug/
3. Update documentation

📁 Files Summary

New files created (Phase 6.1):

- ✅ `events/event_bus.py` (528 lines)
- ✅ `events/event_types.py` (454 lines)
- ✅ `events/spatial_index.py` (453 lines)

Files to modify (Phase 6.2):

- 🔧 `events/event_bus.py` - Integrate spatial index
- 🔧 `simulation/execution/simulation_loop.py` - Initialize event bus
- 🔧 `agent/story_driven_agent.py` - Subscribe to events
- 🔧 `scenarios/dynamic_policy_engine.py` - Publish policy events
- 🔧 `simulation/infrastructure/infrastructure_manager.py` - Publish failure events

Total changes: ~5 files, estimated 200-300 lines of new code

✅ Status

Phase 6.1: ✅ Complete (Event bus + pub/sub working) **Phase 6.2:** 🟡 Ready to start (Spatial index created, integration pending) **Phase 7:** ⏸️ Waiting (Dynamic replanning - requires Phase 6.2)

This is your roadmap for Phase 6.2! Shall we start with updating event_bus.py to use the spatial index?